

Task 1. 数据类型与运算基础

1. Java 的八种基本数据类型分别是什么？

- 字节型:byte
- 短整型:short
- 整型:int
- 单精度浮点:float
- 长整型:long
- 双精度浮点:double
- 字符型:char
- 布尔型:boolean

2. 四种整型类型各占多少字节，表示范围大致是多少？

byte:1 字节, $-2^7 \sim 2^7-1$

short:2 字节, $-2^{15} \sim 2^{15}-1$

int:4 字节, $-2^{31} \sim 2^{31}-1$

long:8 字节, $-2^{63} \sim 2^{63}-1$

3. 解释基本类型与引用类型的区别。

基本类型:直接把真实的值存到变量里

引用类型:把内存地址存到变量里面,当两个变量都指向同一个地址时,改变其中一个另一个也会跟着改变

```
int a = 4;
char c = '0';
int b = a + c;
```

4. 这里发生的是隐式类型转换还是显式类型转换？

- **隐式转换:**把字符c自动转换为整形

5. b 的值是多少？请解释为什么。

- b的值为52
- 我先使用 `System.out.println(b)` 输出b发现是52而不是4,我就去问ai,得知字符质上是用数字编码存储 (ASCII 码) ,其中"0"的ASCII码是48

再执行下面这段代码：

```
int a = 5;
int b = 7;
int c = (++a) + (b++);
System.out.println(c);
System.out.println(a + " " + b);
```

6. 写出运行结果，并解释每一步计算过程。

- 运行结果: 13 6 8
- 计算过程:
 1. $a=5, b=7$
 2. `++a`:**自增前置**,先 $a+1$ 再取出变量 $a=6$
 3. `b++`:**自增后置**,先取出 $b=7$,再 $b+1$

4. 因此 $c=6+7=13$, $a=6$, $b=8$

7. 为什么“两个正数相加结果可能变成负数”? 请从二进制存储和溢出的角度解释。

- java的int类型是**32位有符号二进制**,最高位符号位的**1表示负数,0表示正数**
- 当两个很大的数相加超过 $2^{31}-1$ 时,最高位**符号位**就会被迫从**0变为1**,所以会变为负数

8. float 为什么可能出现精度问题? 你可以简单说明, 不要求推导完整标准。

- float只能用**2的负整数次幂**进行小数的拼接,类似**0.1和0.2**这类小数只能存储近似值

挑战

```
//第一组
Integer x = new Integer(18);
Integer y = new Integer(18);
System.out.println(x == y);
//第二组
Integer z = Integer.valueOf(18);
Integer k = Integer.valueOf(18);
System.out.println(z == k);
//第三组
Integer m = Integer.valueOf(300);
Integer p = Integer.valueOf(300);
System.out.println(m == p);
```

- 我将代码分成三组分别解释

1. 第一组: new 表示在**堆**中创建一个**对象x**和**对象y**
输出 **$x==y$** 的布尔值,比较两个对象的**地址**
显然两个不同对象的地址不相同,所以返回结果为**false**
2. 第二组: valueOf 表示从缓存池中拿取**对象的地址**
由于k,z两次调用 valueOf,所以拿取到的是**同一个对象的地址**
所以返回结果为**true**
3. 第三组:首先要知道**静态缓存数组**的范围是-128~127
调用 valueOf(300) 超出了缓存范围,自动**生成新对象**
因此m和p是两个不同对象,**同理第一组**,所以返回结果为**false**

Task 2. 控制流与基础算法

2.1 if-else 与 switch

补全函数:

```
static boolean isLeapYear(int year) {
    return (year %4 ==0 && year%100 !=0 ) || (year%400 ==0);
}
```

1. switch-case 和 if-else 的适用场景分别是什么?
 - switch-case:适用于条件是**离散数据**类型判断
 - if-else:适用于条件是**范围或复杂逻辑**的判断
2. 有人说 switch-case 是 if-else 的“语法糖”, 你是否同意? 请说出你的理由。
 - **不同意**
 - if-else是顺序查找,时间复杂度是 **$O(n)$**
 - switch-case是**跳转查表**,时间复杂度是 **$O(1)$**

2.2 循环

```
static void print(int n) {  
  
    for (int i = 0; i < (n + 1) / 2; i++) {  
  
        for (int j = 0; j < (n-1)/2 - i; j++) {  
            System.out.print(" ");  
        }  
  
        for (int k = 0; k < 2*i + 1; k++) {  
  
            if(k == 0 || k == 2*i){  
                System.out.print("*");  
            }else{  
                System.out.print(" ");  
            }  
        }  
        System.out.println();  
    }  
}
```

2.3 递归与迭代

1. 递归

```
public static int fibonacci(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    return fibonacci(n-1) + fibonacci(n-2);  
}
```

2. 迭代

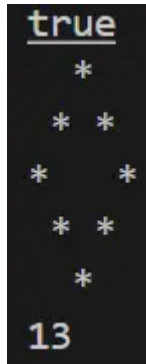
```
int fibonacci(int n) {  
    if(n == 0) return 0;  
    if(n == 1) return 1;  
    int a = 0; // F(0)  
    int b = 1; // F(1)  
    int res = 0;  
    for(int i = 2; i <= n; i++){  
        res = a + b;  
        a = b;  
        b = res;  
    }  
    return res;  
}
```

1. 递归和迭代的核心区别是什么？

- **递归**:函数自己调用自己
- **迭代**:利用循环执行,变量保存状态,不断更新变量

2. 为什么很多场景下更偏好迭代？

- **时间复杂度**: 以上面的斐波那契数列为例, **递归**的时间复杂度是 $O(2^n)$, 而**迭代的**只有 $O(n)$
 - **调用栈**有溢出风险: **递归深度**过大会耗尽**调用栈**
3. 循环是否总能被递归取代? 如果能, 代价是什么?
- 可以完全取代
 - 上文也说到有**栈溢出**风险, 而且**逻辑部分**不易读懂



汉诺塔

```
public class hanoiDemo{
    public static void hanoi(int n,char from,char aux,char to ){
        if(n==1){
            System.out.println("移动圆盘"+from+"->"+to);
            return;
        }
        hanoi(n-1,from,to,aux);
        System.out.println("移动圆盘"+from+"->"+to);
        hanoi(n-1,aux,from,to);
    }
    public static void main(String[] args) {
        // A柱n块, 借助B, 移动到C
        hanoi(3, 'A', 'B', 'C');
    }
}
```

Task 3. 异常处理与程序健壮性

- Error 和 Exception 的区别是什么? 面对它们时, 程序的处理态度应该有什么不同?
 - **Error**是JVM本身的**严重错误**, 不是代码的问题
处理态度: 终止程序, 排查环境和内存
 - **Exception**是**应用层面**的错误
处理态度: 通过 `try-catch` 捕获, 并让程序继续运行, 或者 **throw** 给上层方法当做**特殊异常值输出**
- checked exception 和 unchecked exception 的区别是什么? 它们通常分别由什么原因引起?
 - checked exception: 必须 `try-catch` 捕获 或者 `throws` 声明抛出, 否则编译报错
 - 编译不强制处理, 都可以编译通过, 属于 **RuntimeException** 子类
- 代码如下:

```
//空文件异常类
public class EmptyFileException extends Exception {
    public EmptyFileException(String message) {
        super(message);
    }
}
```

```

    }

}

//计算均值类
public class Avgcalculator {
    public static void main(String[] args){

        String filepath = "data.txt";
        double avg = 0;
        int sum = 0;
        int count = 0;
        try (BufferedReader br = new BufferedReader(new FileReader(filepath))) {
            String line;
            //提取字段
            while ((line = br.readLine()) != null) {
                line = line.trim();
                if (line.isEmpty()) {
                    continue;
                }
                //转为int类型
                sum += Integer.parseInt(line);
                //一共多少行
                count++;
            }
            if (count == 0) {
                throw new EmptyFileException("The file is empty.");
            }
            avg = (double) sum / count;
        } //IO读取异常
        catch (IOException e) {
            System.out.println("Error reading the file: " + e.getMessage());
        } //字符串不能转整数异常
        catch (NumberFormatException e) {
            System.out.println("Error parsing a number: " + e.getMessage());
        } //自定义空文件异常
        catch (EmptyFileException e) {
            System.out.println(e.getMessage());
        }
        System.out.println(avg);
    }

}

```

Task 4. Stream 流

先观察这段代码：

```

List<String> strings = List.of("I", "am", "a", "list", "of", "Strings");
Stream<String> stream = strings.stream();
Stream<String> limit = stream.limit(4);
System.out.println("limit = " + limit);

```

1. 直接打印 limit，会得到什么类型的结果？
 - **输出结果**: java.util.stream.SliceOps\$1@7344699, 是一串**哈希值**
2. 为什么它不会直接把流里的元素打印出来？

- Stream流分为**两种方法**,中间方法,和终止方法

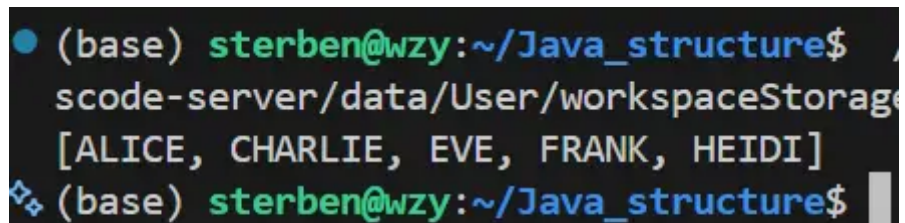
中间方法: `.stream()`、`.limit()`、`.filter()`、`.map()` 等不会**遍历数组**只是组装过程

终止方法: `.forEach()` `.collect()` `.count()` 等才会**遍历数组**执行操作

3. 流操作:

```
List<String> passingStudents = (List<String>) students.stream()
    // 在这里补充你的流操作
    //过滤分数大于等于 60 的学生。
    // 把姓名转换成大写。
    // 按姓名字母顺序排序。
    // 收集成 List<String> 并打印结果。
    .filter(s -> s.getScore() >= 60)
    .map(s -> s.getName().toUpperCase())
    .sorted()
    .toList()
    ;

System.out.println(passingStudents);
```



```
● (base) sterben@wzy:~/Java_structure$ java -cp . code-server/data/User/workspaceStorage/sterben/Java_structure/Java_structure.jar
[ALICE, CHARLIE, EVE, FRANK, HEIDI]
❖ (base) sterben@wzy:~/Java_structure$
```